淘宝动效解决方案分享

原创 跨端技术团队 大淘宝技术 2026年4月15日 01:37 浙江 23人



本文提出了一套平台化、协议化、工程化的动效解决方案，覆盖设计（AE插件）、编辑（可视化画布SDK）、布局（align/group动态对齐与成组）、播放（H5/Weex跨端统一Player）、压缩（二进制优化）、代码生成（Lottie→Anime.js）及AI辅助（MCP协议动效Agent）全链路，解决碎片化、多端不一致、性能差、维护难等痛点，实现“一次制作、多端复用、智能可控”。



在业务迭代中，动效越来越多地承担「表达品牌、引导转化、承载营销玩法」等职责；若工程侧仍停留在「能播动效」这一层，往往在扩展能力、多端一致性、上线质量与长期维护上反复踩坑。下面归纳我们在实践中反复遇到的问题，并说明为何要走向平台化、协议化、工程化的动效解决方案。

1. 标准 Lottie 很好，但业务上常缺以下几类扩展能力，例如：
   - 布局与结构：响应式布局、动态元素居中、动态元素成组等在运行时可动态调整；
   - 动态内容：资源快速替换、局部热更新而非整段重播、按接口/状态驱动某一图层或切片；
   - 降级与容错：图层级别、资源级别的降级策略（例如关动效保主流程），避免复杂动画拖垮页面或引发崩溃；
   - 可编排与可检索：元数据、查询标签、命名规范，方便业务按规则检索片段驱动动画；
2. 碎片化方案的中长期成本过高

   各业务线对于动效的能力要求不一，采用的播放器种类繁多，有专属于业务的定制诉求，不方便复用；而且在某些场景会各自采用hack实现，短期能上线，长期可能会加大维护成本；动效这种跨团队、跨端的共性能力，适合沉淀为平台级供给。
3. 「能导出」不等于「能上线」

   从 AE 到线上，中间缺的是：检测与优化、可视化编辑、动态替换与布局、降级与监控、多端一致播放。缺一环可能会在联调与线上事故里补票。

4. 当播放器在Weex与H5的能力不对等时，业务无法共用一套动效资产

   当两端播放器能力不一致（如API、扩展协议、降级策略、替换能力不同）时，会出现同一需求采用多套动效资源、多套方案播放的情况， 平台侧无法承诺「一次制作，多端复用」。我们希望 H5 / Weex 采用同套 API 与扩展协议，同套资源同套代码能多端复用。

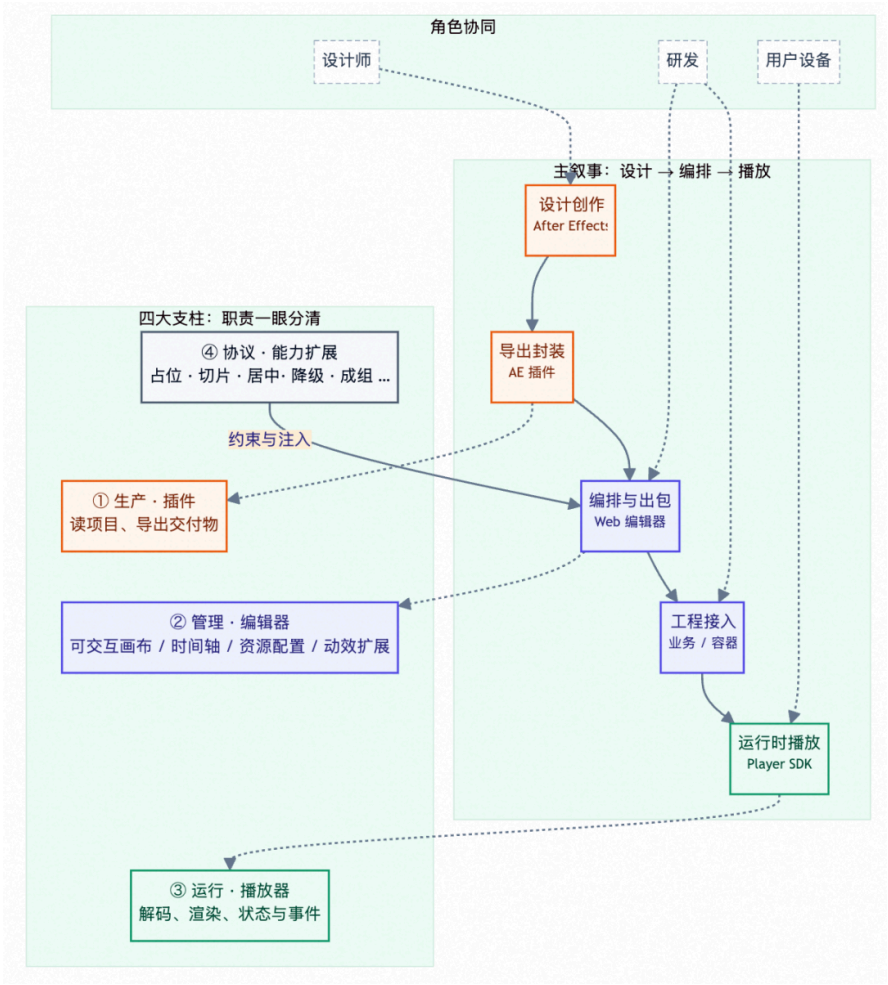
5. 体积与性能是持续压力

   例如动效文件压缩与解析（动效体积、图片/矢量资源是否可进一步优化和压缩）、文件渲染性能（帧率、首帧时间、内存峰值）等，属于全链路问题，单靠某一端的临时优化很难根治。

6. 易被忽略但会放大成本的内容

   资产版本管理、与发布的衔接；可校验的设计—研发约定，如图层命名、占位符约定、切片语义，需要可校验的规范，减少口头对齐；部分场景的无障碍（动效开关、降级策略等）。

为此，我们提供了以下的动效解决方案，用一条清晰的交付主链路，把「设计产出 → 可配置资产 → 运行时播放」都串联在了一起；同时在中间层用扩展协议来承载占位、切片、降级、编组、布局等策略，避免把业务规则硬编码进播放器或散落各处。



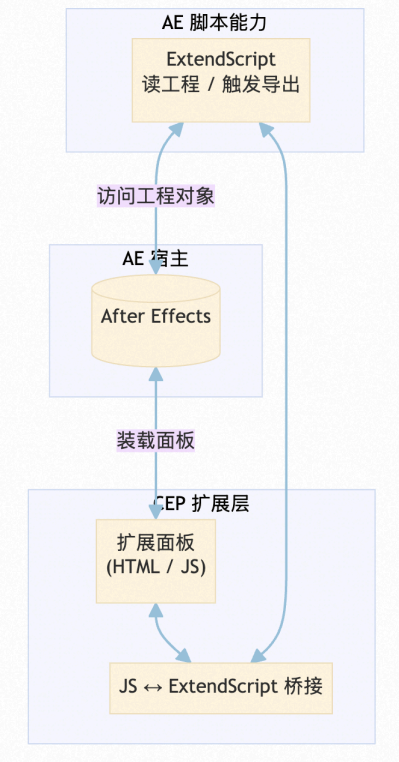
1. 对设计师来说，仍在 AE 里主创作，交付方式升级为链接 + 平台能力；预览、标注、体积与兼容性提示前移，减少「设计以为能播、上线发现不能播」的返工；

2. 对开发者来说，在编辑器里完成二次编辑与业务配置，提供动效扩展能力，不必手写DOM hack；统一播放器 API 屏蔽多端差异，配合错误类型与降级资源，接入更方便；
3. 对业务与平台来说，一套链路覆盖生产、交付、运行与治理，便于规模化复用与质量保障。

02 技术点分析

AE插件研发

在研发AE（After Effects）插件的过程中，我们沉淀了插件在插件开发、插件热更新、插件日志、读取AE数据、插件构建、构建打包、版本推送方面的全流程能力。基于 CEP（Chromium Embedded Framework） 的 AE 扩展：面板是 Web 技术栈（React），与 AE 本体通过 ExtendScript / JSX 交换工程数据；对Bodymovin 进行了二次开发与交付。



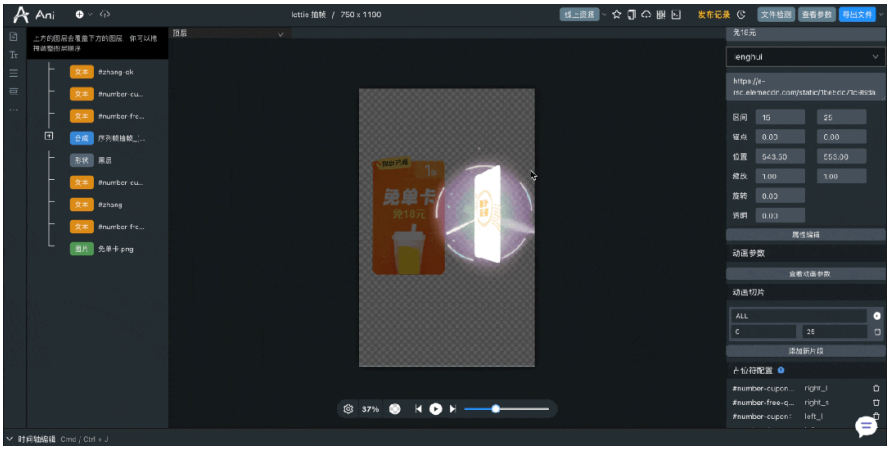
通过此插件我们打通了AE软件与动效平台的直接连接，设计师可通过此插件提前查看动效还原度、进行动效检测与优化，同时直接在动效平台完成动效交付。此插件提供了以下的能力：

- 提前进行扫码预览，保证还原度
- 图片资源编辑与一键优化
- 提前进行资源检测、体积检测、兼容性检测
- 与动效平台集成，直接交付动效
- 可以查看并交付动效参数

设计师无需手动导出文件、预览文件、交付文件、上传文件，只需要分享项目空间给到研发同学即可。

动效可视化编辑

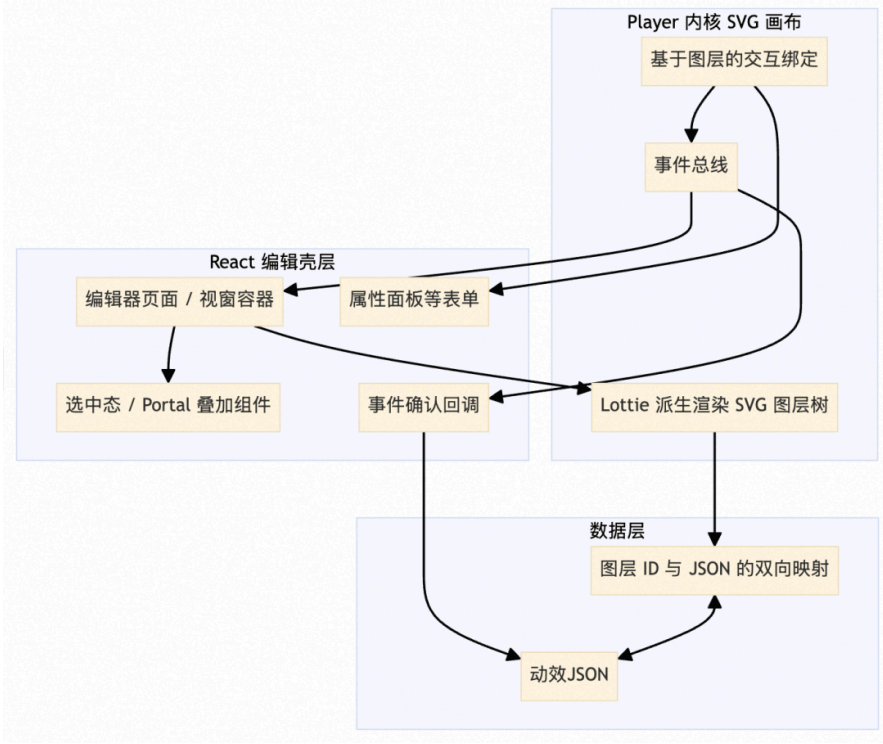
动效编辑器赋予开发者与设计师更多的灵活性，能对导入的动效进行细致的调整，而不只是简单的使用已经做好的动效。在中间动画播放器对应的画布中直接拖动到动画对应的某帧，选择动画中的元素，调整位置、尺寸、属性，或者为其添加占位符、定义降级策略等等。



在浏览器里编辑动画，本质上是两件事叠在一起：

1. 渲染层：把动画变成可看见的 SVG（以及与之对齐的坐标系）；
2. 编辑层：在「看起来像画布」的区域里，让用户像在设计工具里一样点选、拖动、改尺寸，并把操作可靠地写回动画数据中。

本项目的做法是：渲染仍走Lottie动效的 SVG 渲染，但在图层创建时注入「可编辑标识」与「交互事件」；React 页面负责容器、选中态、叠加 UI 与历史记录。 这样避免用一套完全自研的渲染器重做 Lottie，又能在图层的粒度上做选中与变换。



预处理过程为每个可编辑图层生成稳定唯一标识，在 SVG 图层根节点上同时挂上：

- 作为 DOM `id`（便于 `getElementById` 精确定位）
- 作为数据属性（便于与侧栏、列表里同一图层对齐）
- 可选的样式类名（区分「画布上的 SVG 」与其它 UI）

渲染器在创建图层容器时，若该图层带有上述身份，则注册到「当前画布上所有可交互图层 id 列表」，并立即挂上交互。这样选中与变换的入口始终是「用户点中的那个 SVG 组」，而不是再做一次几何拾取去猜 JSON 里的第几个图层。



用户点击图层时，通过事件总线向外壳抛出 「某图层被选中」 事件，外壳（编辑器）监听该信号后，把当前选中图层 id 存进 React 状态，驱动后续 UI。

若每次鼠标移动都深拷贝并改写整份 JSON，会卡顿且难以做惯性、节流。因此交互上采用经典策略：

- 拖动中：只改当前 SVG 节点上的 二维变换（例如 `transform` 矩阵），保证流畅度；
- 拖动结束：计算相对空间的总位移，通过统一 变换动作交给数据层，按规则写回 JSON；有动画则按关键帧结构批量偏移等。

画布显示比例不是 1:1 时，位移总量要除以编辑器缩放系数，才能得到实际的偏移长度单位。

缩放过程中同样先改矩阵与内部位图尺寸相关属性，实时向面板派发缩放系数等；结束时发出 resize 类动作，由数据层更新 JSON 中缩放与资源展示相关字段。

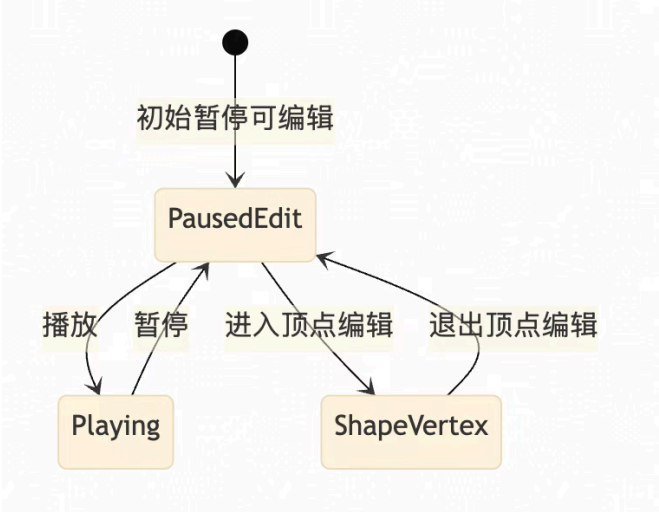




为保证「看动画」与「改动画」“不打架”，用事件总线区分模式：

- 进入播放：若存在选中，先清理选中样式；并移除所有已注册图层的指针交互（避免动画播放时误拖）；
- 暂停：重新为各图层加回交互，恢复编辑能力；
- 进入矢量顶点编辑：临时移除常规图层交互（避免与顶点拖拽冲突），退出时再恢复暂停下的编辑能力。

这是一个简单的状态机，保证同一时刻只有一种状态：



选中后，需要在画布上画选框、控制点、辅助线，若放在 React 树顶层用绝对定位计算，容易与 SVG 变换、滚动、缩放错位。

我们的思路是，用 Portal 把叠加组件挂到「当前选中图层根节点」内部。这样叠加图形与图层内容天然同一 SVG 坐标系，只需按图层的几何信息（如宽高、路径顶点）生成子 `g` 与 `path`、`circle` 就行。

- 普通选中框：例如针对可缩放的图片层，根据图层矩形生成描边与控制点（具体展示策略由数据类型决定）；
- 矢量顶点模式：从 JSON 读出路径顶点，在相同 `g` 下渲染可拖顶点，拖动时用交互库更新顶点坐标，松手后写回 shape 数据。

最后修改完成后，内核通过事件向上抛出动作（移动、缩放、进入/退出顶点编辑等）。外壳监听后：

1. 调用更新工具（内部通过 id 映射找到图层或资源）
2. 更新应用状态中的当前 JSON 文件
3. 触发重新加载视图，使画布与数据一致

历史记录（撤销栈）在动作已落成新 JSON 时追加，保证一步操作对应一份可还原快照。

另一条通道是外部直接改 JSON（如撤销、批量替换）：通过同一事件总线通知播放器替换数据，保持内核与外壳一致。

核心的几点考虑整理如下：

描述	设计要点
身份绑定	JSON 图层稳定 id → SVG 根节点 id + 数据标记，作为一切交互与回写的锚点
选中	点击上行事件 + 外壳状态；可选 DOM 类名同步侧栏
变换	拖动/缩放先改 SVG，结束再写 JSON；位移用缩放系数还原设计单位
叠加 UI	Portal 进选中图层，共享坐标系，减少换算误差
模式	播放与编辑互斥；顶点编辑再互斥一层
桥接	全局对象挂载事件总线、当前帧、缩放系数、选中 id
属性面板	高频过程可命令式写表单，降低重渲染压力

我们将其抽离为了动效的可交互式画布SDK，可以让你的动效编辑起来更简单。

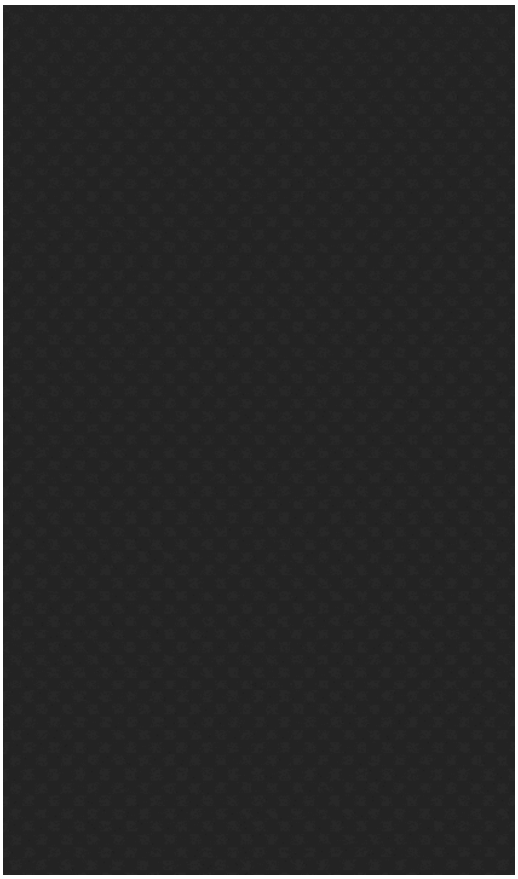


动效内容布局约束

居中对齐（align）和成组布局（group）是用于处理动画中动态内容替换后布局调整常用到的功能。当业务通过 `replaceData` 替换文本或图片内容时，这些元素的宽度可能发生变化，导致原始布局错乱。居中对齐功能确保一组元素在容器内保持整体居中，而成组布局功能则确保同组元素根据对齐方式自动调整相对位置，维持视觉一致性。

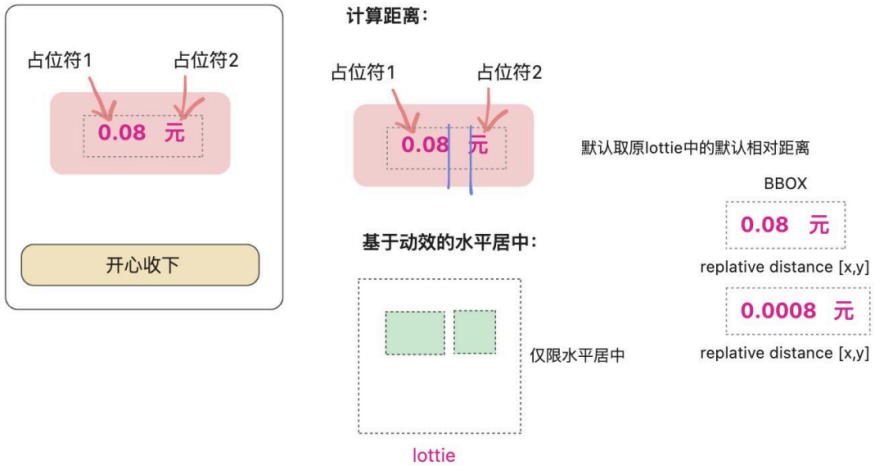
- 动画居中对齐

例如中间的金额数字是动态传入的文案，这里文案的长度不固定，比如无论是“15.99元”或者是“100元”，都希望能整体居中对齐。



之前业务针对这类的需求，会采用hack方式去实现：让设计师给到图层id，研发去查询指定的DOM，并重写innerHTML，非常简单粗暴。我们希望在编辑器内能配置占位符元素的居中，包括单个占位符的整体居中、多个占位符组合后的整体居中。不管在播放器中占位符传入多长的文本，都能自动调整其位置。

1. 编辑器：能配置【N个占位符】的对齐模式



2. 播放器：需要识别【N个占位符】的对齐模式

单元素处理逻辑如下：

```
parentOffset = 父链上 sum(p - a)（若有）
若存在 parent:
  p.x = parentOffset - pageW/2 - anchor.x   // 与「父级 + 锚点」关系推导
否则:
  p.x = pageW/2 + anchor
```

多元素（N ≥ 2）对应的通用范式：

先将图层按 `axis` 排序，再算原始左右边距 `minL`、`maxR`（相对 `pageW`），再用 replaceData 得到新宽度 `newW[ ]`，更新 `minL`/`maxR`。

保持相邻间距（文案变宽时不是简单整体平移）：对首尾元素施加 `remainGapOffset1/2`，中间元素在 N>2 时尽量不动，由首尾吸收宽度差。通用范式可描述为：

```
组合居中的条件：
计算中轴：
      c1 = p1 - a1
      c2 = p2 - a2
计算整体居中的位移 alignCenterOffset
      minL = c1 - w1/2  minL = c1 - new1/2
      maxR = lottieW - (c2 + w2/2)  maxR = lottieW - (c2 + newW2/2)
      alignCenterOffset = (maxR - minL) / 2
计算保持间距不变的位移
      addW = (newW1 + newW2) - (w1 + w2)
      remainGapOffset = addW / 4
计算父节点的位移
      parentOffset = parentP1 - parentA1
计算p：
      p1 = p1 - remainGapOffset + alignCenterOffset - parentOffset
      p2 = p2 + remainGapOffset + alignCenterOffset - parentOffset
扩展到N个元素的通用范式：
p1 += -remainGapOffset1 + alignCenterOffset - parentOffset（第一个元素）
p2 += alignCenterOffset - parentOffset（中间元素）
p3 += alignCenterOffset - parentOffset（中间元素）
p4 += remainGapOffset2 + alignCenterOffset - parentOffset（最后一个元素）
```

整体流程如下：

1. 一维轴上排序
2. 每元素半宽得到包围盒左右界
3. 替换内容后更新宽度并重算包围盒
4. 用「首尾补偿」保持间距不变
5. 整体平移使包络在 pageW 内居中
6. 叠加父级/预合成偏移

- 动画成组布局

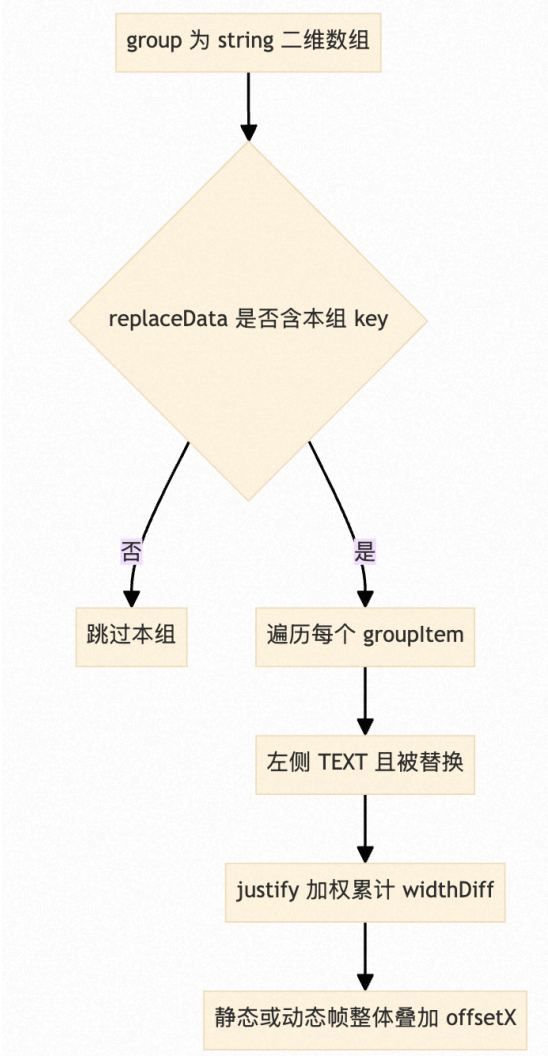
例如在动画的播放过程中，数字“160”是动态传入的，数字的长度不固定，那么后面的“积分抽”、“20”、“次”都要左右进行适当的偏移，才能与前面的数字保持相同的间距。



例如最前方的文案可能是“获得”，也可能是“今天续卡再送”，中间的数字也是不固定的，后面的文案也是不固定的，那完全可以将这些元素采用成组的方式，来保证元素间的间距始终一致。



`group` 不负责把一组东西摆到屏幕中央，而是：在同一组占位符里，只关心「左侧谁变宽了」，按文本对齐方式决定当前层要额外平移多少，避免右侧被左侧替换后的文字盖住或留白异常。



对当前元素 `groupItem`：

1. `currentInitPosition = getInitPosition(当前层)`（首关键帧或静态）
2. `leftItems` = 同组内满足：`ty === TEXT` 且 `replaceData[item]` 有值且初始位置在当前左侧的项
3. 对每个 `leftItem`：

```
widthDiff = newWidth(leftItem) - oldWidth(leftItem)
offsetX += widthDiff * justifyFactor(justify)
```

对齐因子（与 AE 文本 `justify` 一致）：左对齐1、右对齐0、居中0.5、两端0。

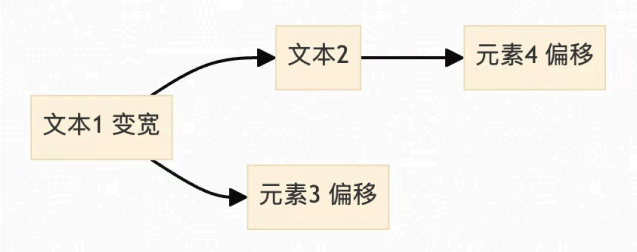
4. 返回 `offsetX`，写到当前层的位置参数

N 元素成组时的通用范式：

设组内从左到右为 `E_0, ..., E_{N-1}`，仅部分有替换数据。对第 k 个元素：

```
offset(k) = Σ_{i<k, E_i 为文本且被替换}  Δw_i × f(justify_i)
```

其中 `f` 由左/右/中/两端对齐决定。以上伪代码的含义：只累计严格在左侧且发生宽度变化的文本，避免全组重排；右侧图层通过不断累加 `offsetX` 实现「右移」。



与 align 的配合关系如下：

- align：全局几何——整组相对画布/预合成的中轴与边距
- group：局部几何——组内相邻关系在动态文案下的增量修正

二者可在同一资源上同时配置，但职责边界清晰：一个管「摆哪儿」，一个管「左边变了右边跟多少」。

跨端动效播放器

在此套动效解决方案中，提供的配套Player是一个跨端动画播放器组件，专为 H5 和 Weex 环境设计。该组件的核心目标是提供一套统一的 API 接口，使开发者能够在不同平台上无缝使用对应动效编辑器支持的一系列具有扩展能力的动画，同时支持动态内容替换、设备降级、性能优化等功能。开发同学只需要几行代码就可以快速接入。

面向业务侧，播放器同时支持声明式与命令式两种使用方式：

- 丰富的 Props：覆盖数据源、自动播放、循环、速度、是否可见即播、替换数据、降级资源、事件绑定等20余配置，多数场景下「几行代码」即可完成接入；
- 无刷新更新：支持在不整段销毁重建实例的前提下更新内容和样式、切片组合、部分播控参数等，减少闪屏与状态丢失，适合列表、Tab、接口回填等多轮更新场景；
- 编程式接口：暴露对实例细节的操控能力（如跳帧、暂停/继续、查询内部状态等），便于与复杂交互的场景。

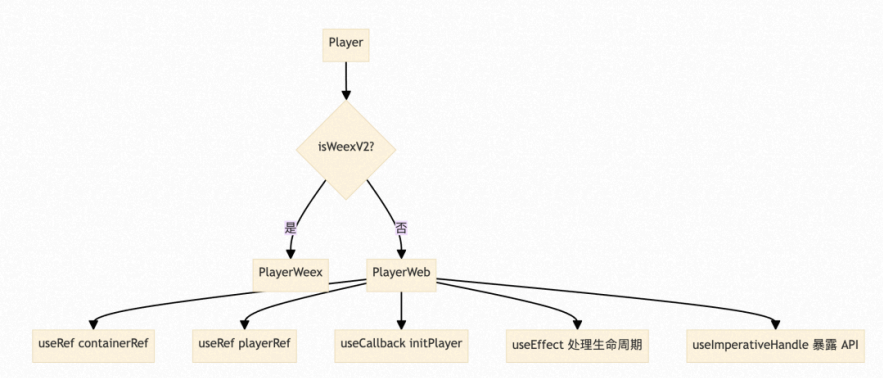
与平台侧扩展协议对齐后，播放器在运行时具备以下能力：

方向	说明
动态内容	占位符替换（文案、图片等）；支持多轮、局部更新
布局	动态多元素 居中（align）、多元素成组跟随（group），与编辑器配置一致
播控	整段播放、按切片（segment） 组合与编排
交互	图层级事件绑定，用于点击热区、埋点、业务跳转等
生命周期	就绪、开始、结束、循环等钩子，便于串联业务流程
稳定性	多级降级（关闭动效、兜底动图等，按业务策略配置）

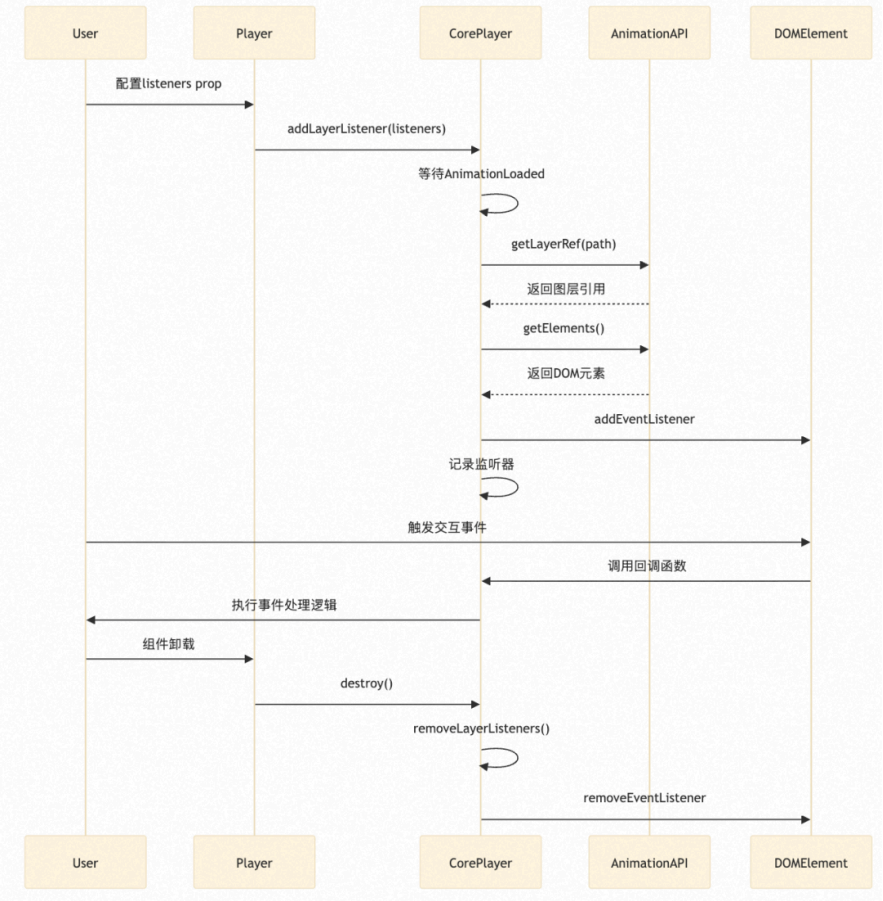
可观测	错误监听，便于区分「数据问题 / 渲染问题 / 配置问题」

播放器在挂载时会先判定当前运行环境，再选择对应的渲染后端：

- H5：走Web侧渲染，便于快速迭代与调试；
- Weex：在容器能力允许时，优先走 原生侧 Lottie 自绘 + 原生组件封装 的路径，以获得更高的还原度、更好的首帧与帧率、以及更可控的内存与 CPU；在容器版本或业务约束下，仍可回退到 Canvas 类过渡方案，保证「能播」与渐进升级。



播放器具备针对动画图层的交互能力，事件处理依赖于准确获取动画中的图层元素：

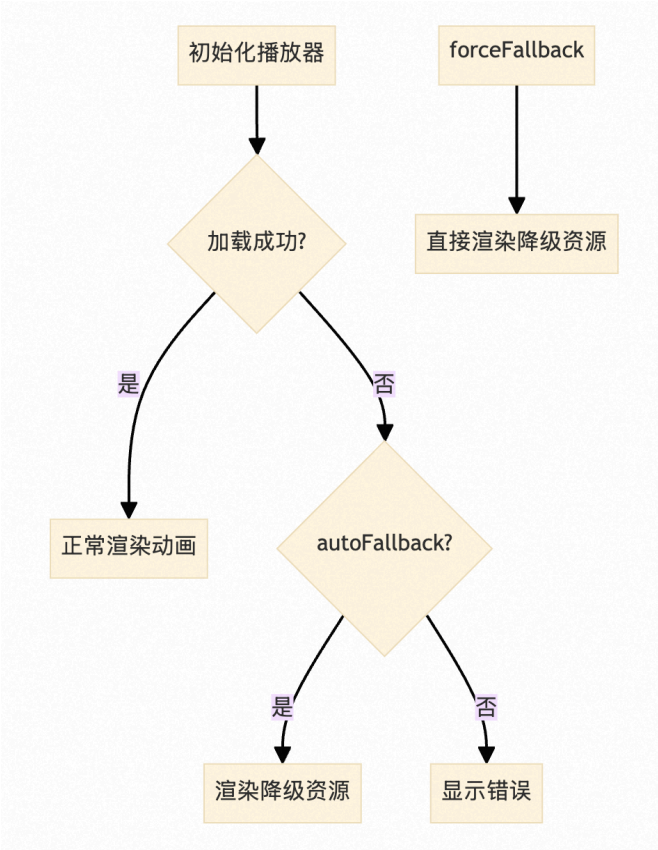


该播放器也提供了以下的错误类型，方便做上层的错误监听和业务处理工作：

错误类型	含义	发生阶段

dataLoadError	动画数据获取或解析失败	拉取/解析资源、初始化数据
renderError	动画渲染或运行失败	降级、加载/渲染失败、实例已销毁等
processError	播放前数据处理不合法	replaceData 等配置校验

对于资源拉取/解析失败/渲染失败的场景来说，也提供了配套的降级资源渲染方案：



播放器具有以下特性：

1. 跨平台一致性：统一 API 接口，屏蔽 H5/Weex 差异
2. 动态内容支持：灵活的占位符替换机制，支持运行时更新
3. 性能保障：多层级降级策略，确保在各种设备上流畅运行
4. 交互增强：图层事件绑定，让动画具备交互能力
5. 开发体验：完善的文档和调试工具，降低使用门槛

这些特性使得此播放器目前能够满足淘宝从简单展示到复杂交互的业务需求。

Weex动效播放

上述的播放器早期主要面向 H5，但在去年及往后集中出现的 Weex 跨端动效需求（如淘宝闪购、互动组件、大促场景等），就无法用同一套资源与同一套播放器承接。

在补齐 Weex 动效播放能力的过程中，我们也曾以 Canvas 作为过渡手段快速验证，但实践中暴露出未知问题较多、性能天花板明显、关键业务能力难以补齐等问题。要满足「还原度、

性能、文本与替换、交互」并长期可维护，需要换一条与 H5 不同、但贴合 Weex 运行时特性的路线：Weex Lottie 走原生侧自绘+ 原生组件封装。

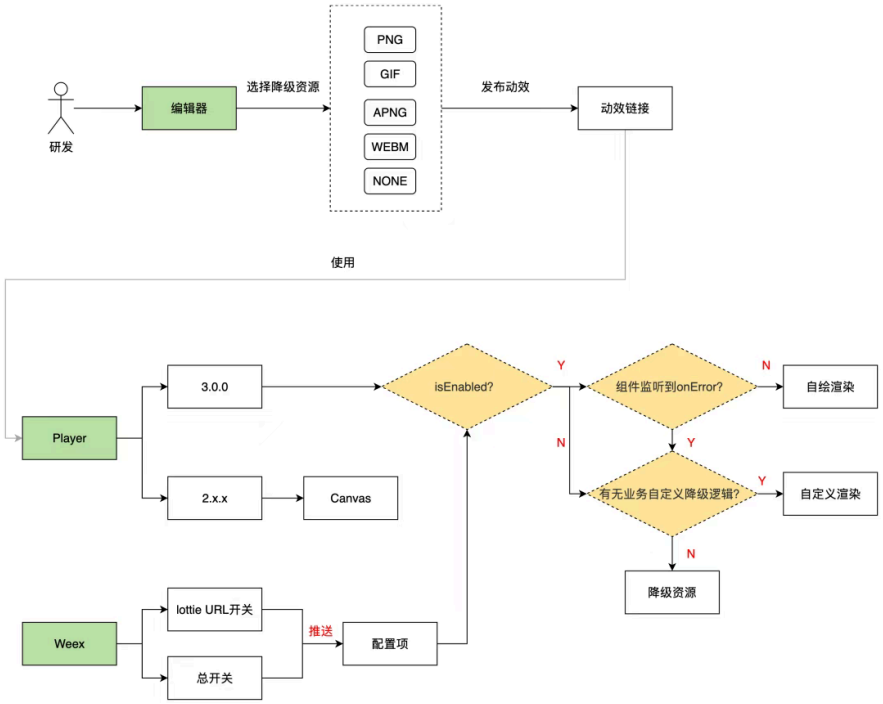
在解决Weex动效问题前，我们也曾考虑过其他方案：

- 在 Lottie-Web 上实现一套 PIXI.Text（或局部 PIXI），但是本质是原生 Canvas API，与 PIXI 整合需改造/重构渲染链路，成本高，且两套底层绘制叠加，性能与问题定位更复杂，仅为文本组件引入完整 PIXI 也偏重；
- 将 Canvas 全量迁移到 PIXI：改造成本极高，Lottie-Web 绘制逻辑需整体迁移，与 Lottie-Web 核心模型不完全一致，功能对齐与回归风险大，依赖与包体积、复杂度上升；
- 在原生 Canvas 上直接绘制文本：性能是历史痛点，社区路径上也曾因性能放弃在 Canvas 上硬扛文本（与当前 Weex 性能目标冲突）。

最终经过考虑，选择了Weex 自绘渲染 + 原生组件形态来支持。两个版本下 Weex Lottie 的能力对比：

Player 2.x（Weex Canvas）	Player 3.x（Weex Dom）
可能存在一些还原度问题	Lottie 还原度高
内存占用较大	内存降低 20% – 50%，CPU 使用率降低 20% – 30%
对于某些 Lottie 可能会有性能问题	大幅优化首帧渲染速度、整体帧率
不能渲染纯文本，仅可渲染文本转形状	纯文本、文本转形状均可渲染
文字仅可被替换为导出了形状的文字	文字可被替换为任意文字
不支持无刷新替换	支持无刷新替换
不支持事件绑定	支持事件绑定

开发如需在Weex环境中渲染动效，则可以采用以下的链路：



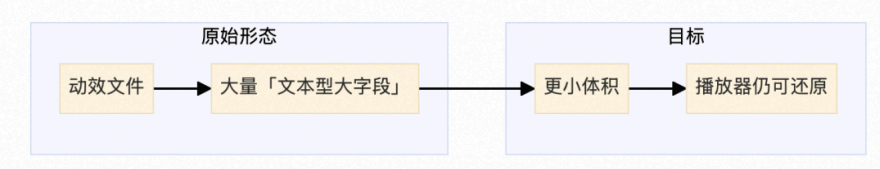
在使用的Weex版本较低、或业务暂时无法升级容器时，2.x 的 Canvas 方案仍有存在价值，可保证可用性；但当业务追求还原度、性能、替换与交互时，应优先评估 3.x 自绘路径，并把 Canvas 视为临时过渡，避免在过渡方案上无限叠加复杂度。

Player在H5和Weex上真正做到了渲染效果和播放器完整能力（包括Props、API、扩展协议等）的对齐。在H5中能正常运行的动效，无需修改一行代码便可在Weex环境中应用。通过Player播放器+Weex Lottie原生组件渲染方案，支撑了淘宝跨端业务中的动效需求：

1. H5与Weex渲染一致：H5和Weex上可保证同等渲染效果；
2. 性能突破：Weex场景能大幅优化首帧渲染速度、整体帧率，内存降低 20% – 50%，CPU 使用率降低 20% – 30%；
3. 业务适配：Weex场景同等支持动态参数注入、动态化修改动效元素、事件交互机制。

动效压缩机制

在动效文件里，往往有一类体积占比很高的文本型数据（常见为base64字符串）。若能在不破坏语义的前提下，让这类数据在文件里占用更少空间，整体包体与传输成本都会下降。



思路：在序列化写入时识别此类字符串，改为按原始字节写入并配上长度信息；反序列化读取时再还原为原来的字符串形式。这样避免把同一信息以「冗长文本编码」重复占空间。

编码侧：识别 → 分支 → 写入

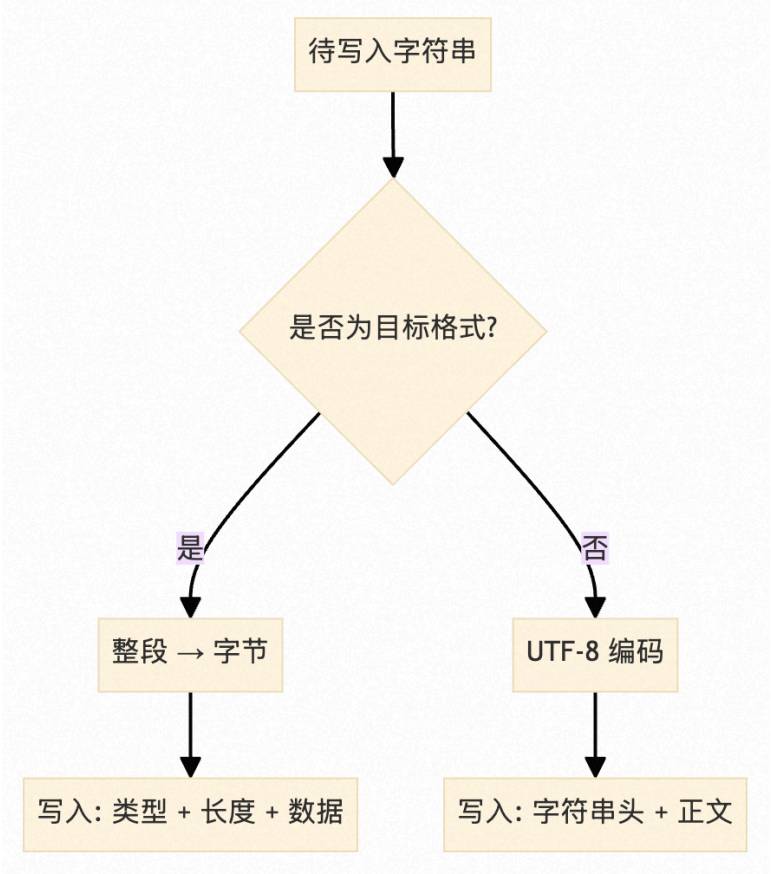
原则：

1. 若某字符串符合「约定的图片数据前缀格式」，则不再按普通 UTF-8 文本逐字展开，而是把整段当作字节序列处理
2. 普通字符串仍走长度前缀 + UTF-8 正文的常规路径

伪代码如下：

```
function 写入字符串(s):
  if s 匹配「约定的数据 URL / 类似形态」:
    将 s 转为字节序列
    调用「写入二进制块」(字节序列)
    return
  计算 UTF-8 长度
  写入「普通字符串」头（含长度）
  写入 UTF-8 字节
```

二进制块写入（示意）：根据长度落在不同区间，写入单字节类型标记与变长或定长长度字段，再追加裸数据。过长则报错或走扩展策略：



解码侧：按类型读回 → 还原字符串

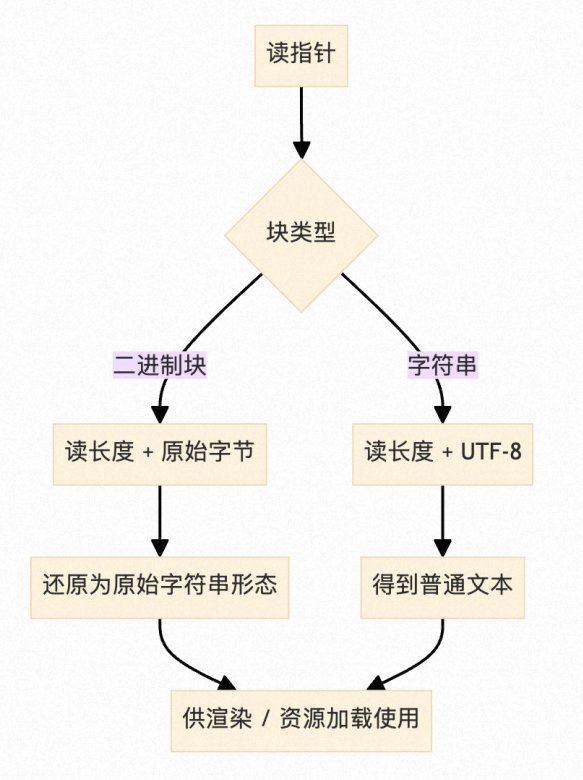
原则：

1. 读取时根据类型标记区分「普通字符串」与「二进制块」；

2. 对二进制块：校验剩余可读长度与上限，读出子区间后，通过与编码时对称的转换还原为原先的字符串（例如仍是那段带前缀的文本）。

伪代码如下：

```
function 读出二进制块并还原为字符串(长度, 头偏移):
  若 长度 超过安全上限: 报错
  若 缓冲区不足: 抛出「需要更多数据」
  定位子数组
  推进读指针
  return 字节 → 与存储时一致的字符串形式（如 UTF-8 解码回原文本）
```



我们采取的对比维度包括：

- 冷启动路径：从初始状态到资源就绪的耗时
- 就绪到首帧：数据可用后到实际绘制的耗时
- 压缩率：整体二进制化后的体积相对原始方案的比例，以及仅针对大字段优化前后的差异

在测试过程中，清空缓存后多次采样取平均，以削弱网络与环境抖动的影响。得到的结论如下：

1. 仅把文本「当成二进制存」而不改识别与分支策略，收益往往有限；对大字段的序列化路径做专门处理后，配合二进制，体积通常能明显下降（经验上可压到原相关体积的约七成以下，具体随素材与结构变化）；
2. 解码还原耗时相对网络加载与 IO通常很小；体积下降带来的加载收益往往大于额外解码成本，因此在该场景下整体仍值得做。

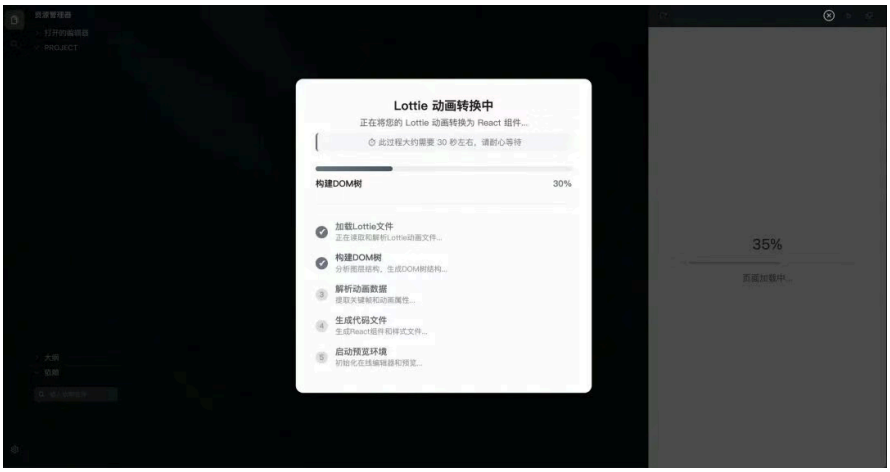
这里我们在编辑器内提供了二进制格式的动效格式，可以导出对应的资源，配合播放器即可使用。

动效代码生成

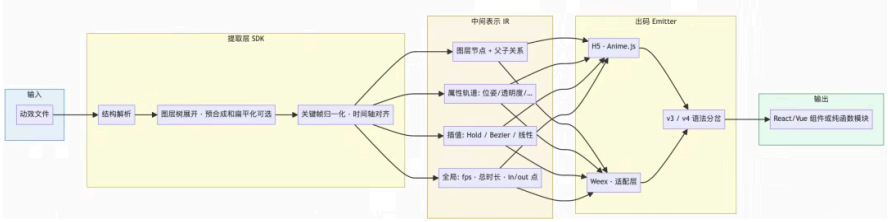
动效平台不仅可以交付动效文件，还可以交付动效代码。要想精准地生成动效对应的代码，我们需要能准确提取动效文件全部动画关键帧信息的能力，于是封装了一个SDK先进行参数的提取工作：



我们可以解析到动画中有哪些图层、图层的基础属性、含有的关键帧动画、关键帧动画的详细信息等等，基于这些参数配合动效框架API（如animejs），可以实现动效文件一键出码。这里我们与香芋同学一起协作，提供了动效出码能力，在项目中可快速引入动效代码：



完整的流程如下：



当前动效 JS 代码具备以下特征，便于在项目中快速引入与维护：

1. 导出形态：默认输出动效组件代码
2. Anime.js 双版本：同一 IR 输出 v3 与 v4 两套实现
3. 双运行环境：同一份代码，支持 H5 与 Weex 播放

在生码的过程中，有几个核心要考虑的点：

1. 层级与变换合成

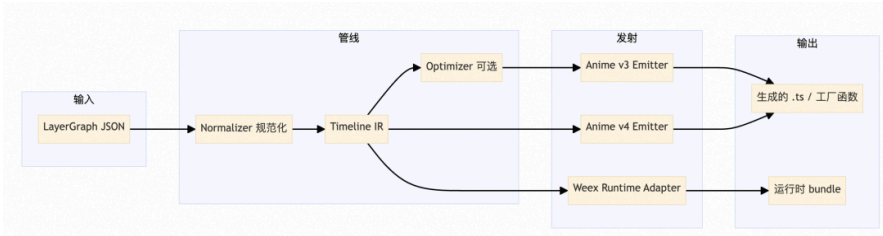
   Lottie 中父层（尤其 `ty: 3` Null）的位移/旋转/缩放会作用到子层。Anime.js 通常作用在 DOM/节点 上，需要在 DOM 嵌套结构或单节点矩阵合成之间做选择
2. 属性语义映射

   抽取数据中的 `s` / `o` / `p` / `r` 等需映射到 anime 的 `scale` / `opacity` / `translate` / `rotate`（或 `transform` 矩阵）
3. 时间与插值

   Lottie 使用帧、`t`、贝塞尔句柄（`i` / `o`）等；Anime.js 使用毫秒、`easing` 或自定义曲线，需要 统一时间轴与插值策略
4. API 分裂

   Anime.js v3（`anime.timeline`、部分选项名）与 v4（时间线、目标写法等变化）需同一套 IR 输出两套 Emitter
5. Weex 约束

   需考虑API适配、样式子集、长度单位、`transform` 支持度等等



动效Agent助手

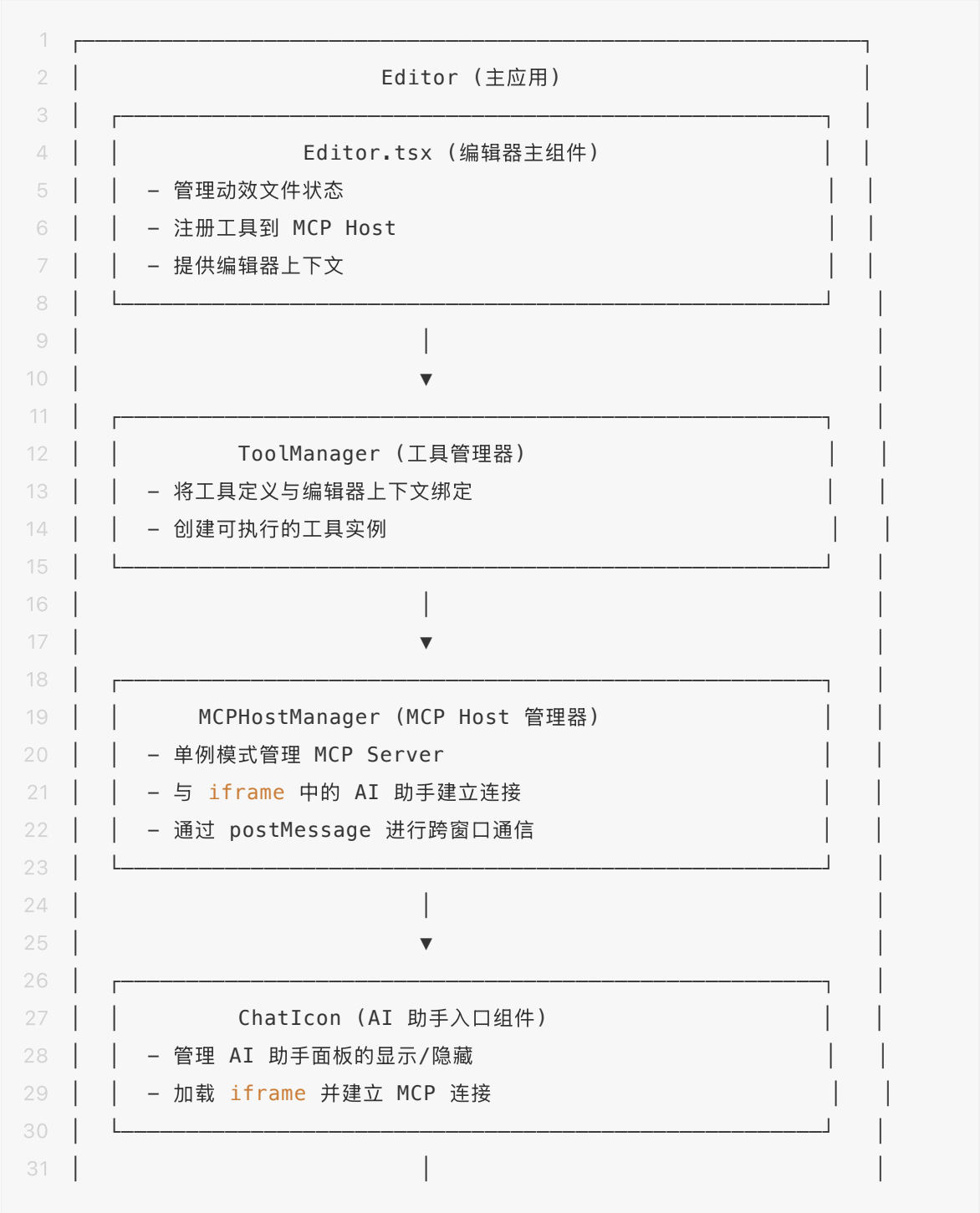
动效Agent助手是一个基于 MCP 协议的 AI 智能助手，集成在动效编辑器中，核心能力如下：

1. 智能答疑：回答用户关于动效生产、动效编辑与播放、性能优化、问题排查等各方面的问题

2. 自动化操作：自动执行编辑器内的操作，如发布文件、配置占位符、设置切片、资源优化等
3. 编辑动效：真正理解动效文件的结构、时间轴、图层、资源、样式、交互逻辑与播放约束，深度理解并直接编辑复杂动效文件
4. 动效诊断：对动效文件进行性能分析、字体问题检测、卡顿问题排查等

动效Agent助手基于我们组无二打造的 zebra-bot 快速集成，它是一个基于 React 和 AI SDK 构建的可配置 AI 助手。项目支持流式对话、工具调用、会话管理、主题切换等核心功能，并通过 MCP服务器管理和 PostMessage 协议实现与外部系统的深度集成。

在编辑器中接入的方案如下：





其中MCPHostManager作为MCP Host 管理器，支持：

- 管理 MCP Server 实例（单例）
- 与 iframe 中的 AI 助手建立连接
- 注册和管理工具定义
- 通过 postMessage 实现跨窗口通信

```
class MCPHostManager {
  // 单例获取
  static getInstance(): MCPHostManager
  // 注册单个工具
  registerTool(tool: ToolDefinition)
  // 批量注册工具
  registerTools(tools: ToolDefinition[ ])
  // 连接到 iframe
  connect(iframe: string | HTMLIFrameElement)
  // 获取连接状态
  getConnected(): boolean
  // 销毁连接
  destroy()
}
```

连接机制如下：

1. 等待 iframe 加载完成（监听 `onload` 事件）

2. 通过 `McpServer.connect(iframeElement)` 建立 postMessage 通信通道
3. 工具调用通过 postMessage 在编辑器主应用和 AI 助手之间传递

工作执行流程如下：

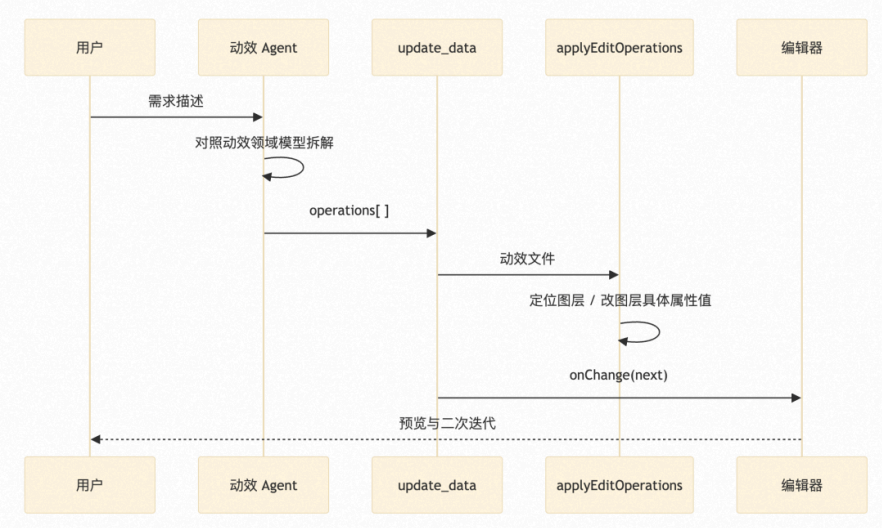
```
用户提问
    │
    ▼
AI 助手理解意图
    │
    ▼
选择合适工具（基于工具描述）
    │
    ▼
构造工具调用参数
    │
    ▼
通过 postMessage 发送工具调用请求
    │
    ▼
MCPHostManager 接收请求
    │
    ▼
查找并执行对应工具 handler
    │
    ▼
工具访问编辑器上下文
    │
    ▼
执行工具逻辑（可能修改编辑器状态）
    │
    ▼
返回执行结果
    │
    ▼
通过 postMessage 返回结果给 AI 助手
    │
    ▼
AI 助手格式化结果并展示给用户
```

要做到深度理解并准确编辑复杂的动效文件，Agent 需要内嵌一套可检索的结构说明 + 可验证的编辑操作。从根合成、时间轴、`ty` 图层类型、`ks` 关键帧、形状/文本/图片、markers、兼容性到「AI 自检清单」，给模型完备的动效知识库。对动效建立的多维理解如下：

1. 结构（动效完整结构）
   - 根级 `w/h`、`layers`、`assets`、`fr`、`ip/op` 定义画布、主时间轴与资源池。
   - 预合成层 `ty=0` 通过 `refId` 指向 `assets` 里带完整 `layers` 的项；改子动画要进 asset 内部，不能只改外层一层。
2. 时间轴（帧是真理）
   - 关键帧时间 `t` 与 `ip/op/st/sr` 共同决定「何时可见、多快播放」。
   - 外层 `ip/op` 与每层 `ip/op` 要求交集；改总时长需同步根与相关层的出点及尾部关键帧。
3. 图层（类型、顺序、父子）
   - `ty` 决定数据挂在哪：形状在 `shapes/it`，文本在`t.d.k`，图片依赖`refId`。
   - 渲染顺序看 `layers` 数组；父子看 `parent` → 另一层的 `ind`。打组、整体位移常通过 Null 父层完成。
4. 资源（实例与定义分离）
   - 图片、预合成在 `assets`；层上 `refId` 必须能在 `assets` 中找到同 `id`。
   - `replace_image`、`scale_layer` 的 `target: 'asset'` 等操作，体现的就是「改显示」与「改素材像素定义」两类路径。
5. 样式（变换与单位）
   - 图层变换：`ks` 下 `a/p/s/r/o`，缩放 100=100%，不透明度 0—100。
   - 形状填充/描边：`fl/st` 的 `c` 常为 RGB 0—1；关键帧链上 `s/e` 维数须一致。
   - 文本：文案与样式多在 `t.d.k[ ].s`，改字串、字号、颜色要连带考虑 `fonts/chars` 与行高等。
6. 交互与播放约束（JSON 能做什么、不能做什么）
   - JSON 没有点击热区；点击由 UI 捕获后调播放器 API。
   - markers 提供命名时间点，供 `seek / 切片。
   - 循环区间多依赖 `playSegments` 等与宿主约定。
   - 播放器与平台能力决定 3D、遮罩、表达式等是否可靠。

从理解动效到编辑动效，工作流如下：

1. 先查再改：先锁定要修改的图层或素材；
2. 拆成有序 operations：拆分原子编辑操作；
3. 自检清单：条目规则（文件是否合法、`refId`、维度一致、文本字体存在、无 parent等）作为 Agent 输出前的的硬检查。



03 写在最后

在当下的业务场景中，很多问题并不存在统一、公认且被广泛验证的标准解法，尤其是在一些跨端、跨场景、跨团队协作的复杂能力建设上，往往是由不同业务线、不同技术团队基于自身诉求，采用各自的方案和技术手段进行“hack式”解决。短期来看，这种方式能够快速满足局部需求，支撑业务上线；但从中长期来看，方案碎片化、能力重复建设、技术债持续堆积等问题会逐渐显现。对于这类具有共性、会被反复遇到的问题，应该从底层能力建设的视角出发，抽象出统一的技术模型、标准化的接入方式，逐步沉淀成平台级的解决方案。只有把零散的“个案处理”升级为稳定的“底层供给”，才能真正降低试错成本，让业务团队把精力更多投入到用户价值和创新表达本身。

在动效场景下，这一问题尤为突出。随着业务对体验要求的不断提升，动效早已不再只是“锦上添花”的视觉修饰，而逐渐成为承载信息传递、状态反馈、情绪表达与品牌感知的交互语言。未来的动效解决方案，不仅需要具备更强的交互能力，能够适配复杂用户操作和多状态流转，还需要在性能、包体积、跨端一致性、可维护性以及生产效率之间取得更优平衡。

进一步看，动效体系的演进不应只是播放器或渲染技术的优化，更应是覆盖全链路的系统化升级，而AI将在其中发挥越来越重要的作用。过去，动效能力链路通常是割裂的：设计生产依赖人工经验，交付需要格式转换和规范对齐，编辑和调试成本高，播放效果受端能力限制，验收环节大量依赖人工比对，上线后缺乏有效监控和质量回收机制。未来，AI有机会在动效全生命周期中持续发力，让整个流程从“生产 -> 交付 -> 编辑 -> 播放 -> 验收 -> 上线 -> 监控”逐步走向真正的无感化与智能化。例如，在生产阶段，AI可以辅助设计生成动效方案、补全关键帧、优化节奏和参数配置；在交付阶段，可以自动完成资源转换、规范检查和多端适配；在编辑阶段，可以通过语言降低任何形式动效类型的调参与修改门槛；在播放阶段，可以结合端能力与运行环境进行动态策略选择和性能优化；在验收阶段，可以基于视觉对比、交互回放和

规则识别自动发现问题；在上线和监控阶段，则可以持续追踪播放成功率、卡顿指标、资源异常、版本回退等关键数据。



本文作者冷卉，来自淘天集团–跨端技术团队。本团队服务于淘宝基础用户产品，是淘宝重要的业务线之一。团队以前端、Weex、Native端的技术解决方案框架和研发模式不断完善自己，持续探索端智能等创新，打造极致的体验和工程技术，保障多端设备的适配和稳定运行，致力于让亿级规模的交付能够更丝滑、更稳定。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术
大淘宝技术官方账号
967篇原创内容
公众号

阅读 1.0万

留言

写留言

还没有留言，来写首评